

Отчёт по лабораторной работе № 5

Название работы: Модель ветвления Git Flow

Студент: Вандышев Андрей Михайлович

Группа: AT-15

Дата выполнения: 11.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Инициализировать репозиторий, создать ветку develop от main		<pre>\$ mkdir lab5_gitflow && cd lab5_gitflow \$ git init -b main \$ echo "# Git Flow" > README.md; git add .; git commit -m "Initial main" \$ git checkout -b develop Initialized empty Git repository in E:/Git/lab5_gitflow/.git/ [main (root-commit) 8597c73] Initial main 1 file changed, 1 insertion(+) create mode 100644 README.md Switched to a new branch 'develop'</pre>
2	Создать feature-ветку login от develop, добавить auth.cfg, смержить с --no-ff		<pre>\$ git checkout -b feature/login develop \$ echo "login: admin" > auth.cfg \$ git add auth.cfg; git commit -m "Add auth config" \$ git checkout develop</pre>

Шаг	Описание	Вып.	Команды / вывод
			<pre>\$ git merge --no-ff feature/login -m "Merge feature/login" Switched to a new branch 'feature/login' [feature/login c15aa1b] Add auth config 1 file changed, 1 insertion(+) create mode 100644 auth.cfg Switched to branch 'develop' Merge made by the 'ort' strategy. auth.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 auth.cfg</pre>
3	Создать feature-ветку db от develop, добавить db.cfg, смержить с --no-ff		<pre>\$ git checkout -b feature/db develop echo "db_host=localhost" > db.cfg git add .; git commit -m "Add DB config" git checkout develop git merge --no-ff feature/db -m "Merge DB feature" Switched to a new branch 'feature/db' [feature/db b23f56e] Add DB config 1 file changed, 1 insertion(+) create mode 100644 db.cfg Switched to branch 'develop' Merge made by the 'ort' strategy. db.cfg 1 + 1 file changed, 1 insertion(+) create mode 100644 db.cfg</pre>
4	Создать release-ветку v1.0 от develop, добавить version.txt, сделать коммит		<pre>\$ git checkout -b release/v1.0 develop \$ echo "version=1.0" > version.txt \$ git add version.txt; git commit -m "Bump to 1.0"</pre>

Шаг	Описание	Вып.	Команды / вывод
			Switched to a new branch 'release/v1.0' [release/v1.0 23fc2f0] Bump to 1.0 1 file changed, 1 insertion(+) create mode 100644 version.txt
5	Слить release в main (с --no-ff), создать тег v1.0	✓	\$ git checkout main \$ git merge --no-ff release/v1.0 -m "Release v1.0" \$ git tag -a v1.0 -m "Version 1.0" Switched to branch 'main' Merge made by the 'ort' strategy. auth.cfg 1 + db.cfg 1 + version.txt 1 + 3 files changed, 3 insertions(+) create mode 100644 auth.cfg create mode 100644 db.cfg create mode 100644 version.txt
6	Слить release обратно в develop (back- merge) с --no-ff	✓	\$ git checkout develop \$ git merge --no-ff release/v1.0 -m "Back-merge release" Switched to branch 'develop' Merge made by the 'ort' strategy. version.txt 1 + 1 file changed, 1 insertion(+) create mode 100644 version.txt
7	Создать hotfix-ветку critical от main, добавить patch.log, сделать коммит	✓	\$ git checkout main \$ git checkout -b hotfix/critical main \$ echo "patch applied" > patch.log

Шаг	Описание	Вып.	Команды / вывод
			<pre>\$ git add .; git commit -m "Critical security fix" Switched to branch 'main' Switched to a new branch 'hotfix/critical' [hotfix/critical 2daf4b8] Critical security fix 1 file changed, 1 insertion(+) create mode 100644 patch.log</pre>
8	Слить hotfix в main (с --no-ff), создать тег v1.0.1	✓	<pre>\$ git checkout main \$ git merge --no-ff hotfix/critical -m "Hotfix critical" \$ git tag v1.0.1 Switched to branch 'main' Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log</pre>
9	Слить hotfix в develop (с --no-ff)	✓	<pre>\$ git checkout develop \$ git merge --no-ff hotfix/critical -m "Merge hotfix into develop" Switched to branch 'develop' Merge made by the 'ort' strategy. patch.log 1 + 1 file changed, 1 insertion(+) create mode 100644 patch.log</pre>

2. Самостоятельное задание

Условие:

Нарисуйте схему Git Flow. Объясните, почему release сливается и в main, и в develop.

Выполнение (ход действий и результат):

```

$ git log --graph --oneline --all --decorate
* 35ab215 (HEAD -> develop) Merge hotfix into develop
|\
* \ c8fd995 Back-merge release
|\
||| * 8cb2f7d (tag: v1.0.1, main) Hotfix critical
||| \
||| /
||| /
||| /
|| * | 2daf4b8 (hotfix/critical) Critical security fix
|| /
|| * 9ccaed0 (tag: v1.0) Release v1.0
|| \
|| /
|| /
| * | 23fc2f0 (release/v1.0) Bump to 1.0
| /
* | 1226018 Merge DB feature
|\
| * | b23f56e (feature/db) Add DB config
| /
* | 59f13d8 Merge feature/login
|\
| /
| /
| * c15aa1b (feature/login) Add auth config
| /
* 8597c73 Initial main
(END)

```

Ветка `release` сливается в `main`, чтобы выпустить новую готовую версию программы в свет. А обратно в `develop` она сливается для того, чтобы мелкие правки (например, обновление номера версии в файлах), сделанные перед самым релизом, не потерялись. Если этого не сделать, дальнейшая разработка пойдет без этих правок.

3. Ответы на контрольные вопросы

1. Вопрос 1:

Флаг `--no-ff` принудительно создает новый коммит слияния. Это необходимо для сохранения истории: так мы всегда видим, где начиналась и где заканчивалась работа над конкретной фичей, даже если конфликтов не было.

2. Вопрос 2:

Технически можно, но это плохая практика. Ветка `develop` собирает готовые изменения от разных участников. Прямые коммиты нарушают изоляцию задач и могут сломать рабочий билд у всей команды. Код нужно писать в ветках `feature`.

3. Вопрос 3:

Если слить хотфикс только в main, ошибка будет исправлена в текущей версии, но останется в исходном коде в develop. При следующем релизе код из develop затрет исправление в main, и баг вернется.

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.